Occlusion-Aware Instrument-Tip Collision Detection via Kalman Trajectory Tracking: Methodology and Measured Evaluation

Abstract

Detecting instrument-tip collisions from a general-purpose object detector is limited, in practice, by the detector's own failures: it frequently loses track of a tip during genuine contact, exactly when the collision signal matters most. This paper proposes and measures an occlusion-aware extension to a prior frame-level pipeline: an independent Kalman filter per tool tip that maintains a position, velocity, and uncertainty estimate through brief detection gaps, feeding a richer feature set (raw Kalman state, derived motion features, and 2-second rolling trajectory summaries) into a classifier, followed by probability smoothing and an explicit event-extraction and one-to-one Hungarian-matching evaluation against annotated collision events. Under an identical, leakage-guarded 5-fold cross-validation protocol with 24 event-preserving time groups and training-side-only threshold selection, adding Kalman tracking and 2-second trajectory history raises Random Forest frame-level F1 from 0.319 to 0.348, and raises F1 specifically on frames where a tip is undetected from 0.448 to 0.504. However, event-level detection coverage falls from 35/64 to 26/64 annotated collisions, while false predicted events fall from 83 to 62. We report this honestly as a measured trade-off - a frame-level and precision improvement, not a demonstrated overall gain in event-level recall - and analyze why.

Keywords

Kalman filtering, occlusion-aware tracking, instrument collision detection, event-level evaluation, Hungarian assignment, cross-validation leakage, procedural training video analysis

I. Introduction

A prior stage of this project (see the companion paper, Sections V-A and V-G) compared eight frame-level and temporal classifiers on 43 hand-engineered features and found that detector occlusion - the tip tracker losing one tool tip exactly during contact - was the dominant limitation, ahead of model choice. This paper tests the direct fix that finding implies: give the classifier an explicit, principled model of tip motion that survives short detection gaps, rather than asking the classifier to work around missing measurements on its own.

We make three contributions. First, a per-tip Kalman filter (Section IV-B) that tracks position, velocity, and estimation uncertainty through occlusion, replacing the ad hoc constant-velocity extrapolation used previously. Second, a revised cross-validation protocol (Section IV-G) that fixes a subtle feature-leakage risk in the original pipeline's globally-computed rolling features, and moves threshold selection strictly onto training-side data. Third, an explicit event-level evaluation (Section IV-J) that converts frame predictions into discrete collision events and scores them against ground truth with one-to-one Hungarian matching, rather than relying on frame-level metrics alone. All reported numbers come directly from a single measured run of this pipeline (results/kalman_updated/), reproducible via the commands in the project README.

II. Related Work

The Kalman filter [1] is the standard recursive estimator for linear-Gaussian state tracking under noisy, intermittent measurements, and is the basis of most classical multi-object tracking pipelines that must survive brief detection dropout. Assigning predicted events to ground-truth events one-to-one is an instance of the assignment problem, solved optimally here with the Hungarian algorithm [2]. The underlying detector (YOLO11n-seg [3], building on the YOLO family [4]) and the gradient-boosted/ensemble classifiers compared here (XGBoost [5], Random Forest [6])

are otherwise unchanged from the companion paper; see that paper's Related Work for detector and classifier background, and for surgical/procedural video analysis context [7], [8].

III. Dataset and Problem Formulation

The dataset is unchanged from the companion study: a single ~13.1-minute pattern-cutting video sampled at 5 fps (3917 frames), with 89 human-annotated events (64 carrying a collision-severity rating, treated as positive) and 558 positive frames (14.25%). This paper reuses the cached YOLO detections from that pipeline (detector_reused=true in protocol.json) - the detector itself is not retrained here; only what happens to its per-frame output is changed.

The problem formulation is extended in two ways relative to the companion paper. First, frame-level prediction is unchanged (y_hat_t = 1 if p_t >= tau), but p_t is now estimated from Kalman-derived features (Section IV-C) instead of, or in addition to, the original 43. Second, an explicit event-level task is added: convert the sequence of frame predictions into a set of discrete predicted events E = {e_1, ..., e_m}, each with a start time, end time, peak confidence time, and confidence score, and evaluate E against the ground-truth event set G with a one-to-one matching under a time tolerance (Section IV-J). Event-level evaluation answers a different, arguably more decision-relevant question than frame-level metrics: not 'how many frames were labeled correctly' but 'how many real collisions did the system flag, and how many false alarms did it raise'.

IV. Proposed Methodology

A. Overall Pipeline

```
Cached YOLO detections (TIPL, TIPR, TIPandCircle per frame)
-> independent per-tip Kalman filter (position, velocity, uncertainty)
-> trajectory/motion/missingness features (raw Kalman state)
[+ optional 2-second trailing rolling-window summaries]
-> Random Forest / XGBoost classifier
-> per-fold, training-side threshold tau
-> 3-frame probability smoothing
-> threshold -> frame-level collision prediction
-> run-length event extraction (merge gaps <= 0.4s)
-> Hungarian one-to-one matching against ground-truth events
-> frame-level, occlusion-conditioned, and event-level evaluation
```

B. Kalman Filter Formulation

Each tip's state is estimated independently as a 4-dimensional vector of position and velocity:

```
x = [x_pos, y_pos, v_x, v_y]^T
```

Between measurements, the state evolves under a constant-velocity motion model with Gaussian acceleration noise. Given a time step dt, the state transition and process-noise matrices are:

```
F = [[1,0,dt,0],[0,1,0,dt],[0,0,1,0],[0,0,0,1]]
G = [[dt^2/2, 0], [0, dt^2/2], [dt, 0], [0, dt]]
Predict: x <- F x, P <- F P F^T + sigma_a^2 * G G^T
```

where P is the state covariance and sigma_a (acceleration_std=100 px/s^2) controls how quickly the filter's confidence in a constant-velocity prediction decays - larger values let the filter adapt faster to real direction changes at the cost of noisier predictions during occlusion. When a measurement z = [x_obs, y_obs] is available, with detector confidence c, the standard Kalman update applies:

```
H = [[1,0,0,0],[0,1,0,0]]
R = I * (measurement_std^2 / max(c, 0.1))
K = P H^T (H P H^T + R)^-1
x <- x + K (z - H x)
P <- (I - K H) P (I - K H)^T + K R K^T
```

Scaling the measurement noise R by 1/confidence means a low-confidence detection is trusted less and pulls the estimate toward it more gently than a high-confidence one - the filter uses the detector's own uncertainty, not just its yes/no output. If a tip has been unseen for longer than max_missing_s=1.5s, the filter discards its state entirely (state -> 'lost') rather than continuing to extrapolate indefinitely; the state's three-way status (freshly detected / predicted-while-occluded / lost) is itself retained as a feature. This is a deliberate, tested design choice - Section V-D shows an example of a gap the filter successfully bridges, and Section VII discusses why longer, unrelated absences (the tip genuinely leaving the frame) are correctly not bridged.

C. Feature Extraction: Three Compared Variants

Three feature sets are evaluated under the identical protocol below, isolating the effect of each addition:

Variant	Feature count	Contents
Original43	43	The companion paper's 43 features unchanged (raw detections, distance/IoU, the original ad hoc constant-velocity tracker a
Kalman	53	Raw detections + Kalman position/velocity/state/uncertainty per tip, kf_distance and its rate of change, relative speed - no ro
KalmanTemporal	85	Kalman features plus 2-second trailing rolling mean/min/max/std for distance, distance rate, per-tip speed, per-tip variance, a

D. Mathematical Feature Formulation

The Kalman-tracked inter-tip distance uses the filter's position estimate for each tip rather than the raw (possibly missing) detection, so it remains defined through brief occlusion:

```
kf_distance_t = sqrt[ (x_L,t - x_R,t)^2 + (y_L,t - y_R,t)^2 ] (Kalman-estimated
positions)
```

Its rate of change (closing/separating speed) and the tips' relative velocity magnitude are:

```
kf_distance_rate_t = (kf_distance_t - kf_distance_(t-1)) / dt
kf_relative_speed_t = sqrt[ (vx_L,t - vx_R,t)^2 + (vy_L,t - vy_R,t)^2 ]
```

Each tip's own speed and acceleration come directly from the filter's velocity state:

```
speed_t = sqrt(vx_t^2 + vy_t^2)
acceleration_t = (speed_t - speed_(t-1)) / dt
```

For the KalmanTemporal variant, a trailing 2-second rolling window is applied to eight of the above signals (distance, distance rate, per-tip speed, per-tip variance, per-tip state), each producing mean/min/max/std - giving the classifier a compact summary of recent trajectory behavior (e.g. 'has this pair been consistently close for the last 2 seconds', not just 'are they close right now').

E. Classification Models

Two models are compared per feature set: Random Forest (300 trees, class_weight='balanced') and XGBoost (250 trees, scale_pos_weight = n_negative/n_positive computed on each fold's training data). Both were used, unweighted, in the companion paper's 8-model comparison; they are retained here as the two strongest-generalizing model families found there. Deep temporal models (LSTM/GRU/TCN) are not rerun in this study (protocol.json: deep_temporal_models_run=false) - with only one annotated video, the companion study already found they underperformed frame-level models, and the revised, stricter protocol here further reduces usable training data per fold, which would only worsen that gap.

F. Class Imbalance

Handled identically to the companion paper: class_weight='balanced' for Random Forest, scale_pos_weight for XGBoost, both computed fresh on each fold's training split only.

G. Cross-Validation and Leakage Prevention

This is the most substantively revised part of the methodology. The timeline is split into 24 groups at ~30-second candidate boundaries, each nudged away from any annotated collision's padded interval (event window extended by 4.5s on each side) so a boundary never lands inside or immediately next to a real event. A 5-fold StratifiedGroupKFold assigns whole groups to folds. Two additional guards address risks the companion protocol did not fully close:

1) Boundary purge. The companion paper's rolling/derivative features (e.g. a 1-second centered rolling mean) were computed globally, sorted by time, before any train/test split - meaning a training frame within that rolling window of a held-out group's boundary could have its feature value subtly influenced by data on the other side of the split. This paper's protocol removes, from each fold's training set, any frame within boundary_guard_s=4s of a currently held-out group's time range, closing this gap for both the legacy features and the new Kalman/rolling ones.

2) Nested, training-side-only threshold selection. Rather than pooling out-of-fold probabilities across all outer folds and picking one global threshold (the companion paper's approach), each outer fold's training data is itself split (a further 3-fold StratifiedGroupKFold) into an inner-fit set and an inner-validation set; the threshold is chosen on the inner-validation predictions only, then the model is refit on the full outer-training set and evaluated once on the untouched outer-test set. No decision that affects a test fold's score is ever made using that fold's own data, directly or through pooling.

H. Threshold Selection

Within each outer fold, thresholds from 0.05 to 0.95 (step 0.01) are swept on the smoothed inner-validation probabilities, and the one maximizing F1 is frozen before the outer-test fold is ever scored. Table II reports the threshold actually selected per fold for the best-performing configuration.

I. Temporal Smoothing

Predicted probabilities are smoothed with a centered 3-frame rolling mean, computed separately within each time group (never crossing a group/fold boundary), before thresholding - unchanged in spirit from the companion paper, applied consistently to every variant here for a fair comparison.

J. Event-Level Collision Detection

Frame-level predictions are converted to discrete events per group: consecutive positive frames are merged into one event if the gap between them is at most merge_gap_s=0.4s; each event records its start time, end time, peak-confidence time, and peak confidence value. Predicted events are matched to annotated collision events by solving an optimal one-to-one assignment (the Hungarian algorithm) over a score matrix where a predicted event scores positively against a ground-truth event only if their time windows overlap within event_tolerance_s=0.5s, with a small tie-breaking bonus for peak-time proximity:

```
score(i,j) = 1 + 0.001/(1+|peak_j - midpoint_i|) if windows overlap within tolerance,
else 0
matches = argmax_(one-to-one) sum score(i,j) (Hungarian algorithm)
```

Unmatched predicted events are false events (analogous to false positives at the event level); unmatched annotated events are missed events (false negatives). This is a substantially stricter and more decision-relevant test than frame-level accuracy: a model could score well on frame metrics while still fragmenting one real collision

into several short predicted events or merging two real collisions into one, both of which the event-level score penalizes appropriately.

K. Evaluation Metrics

Frame-level Accuracy, Precision, Recall, F1, ROC-AUC, and Average Precision are computed exactly as in the companion paper (see its Section IV-J for the formulas). Two additional analyses are reported here: (1) the same metrics computed separately on frames where at least one tip is undetected versus frames where both are visible, directly testing whether Kalman tracking helps the occluded case specifically; and (2) event-level Precision, Recall, and F1 (detected/(detected+false), detected/(detected+missed), and their harmonic mean), plus mean absolute error of matched events' start time, end time, and peak-vs-annotated-midpoint time.

V. Experimental Results

A. Frame-Level Comparison

Variant	Features	Precision	Recall	F1	ROC-AUC	Avg. Prec.
Original43_RF	43	0.2187	0.5914	0.3193	0.6937	0.2558
Original43_XGB	43	0.1949	0.6165	0.2962	0.6143	0.2060
Kalman_RF	53	0.1773	0.6595	0.2795	0.6957	0.2715
Kalman_XGB	53	0.1981	0.5914	0.2968	0.6210	0.2061
KalmanTemporal_RF	85	0.2319	0.6953	0.3478	0.7058	0.2943
KalmanTemporal_XGB	85	0.2257	0.6022	0.3283	0.6613	0.2716

Table I. Frame-level metrics, all six variants, revised protocol (computed directly from results/kalman_updated/metrics.json). KalmanTemporal+RF has the highest frame-level F1 and ROC-AUC of the six.

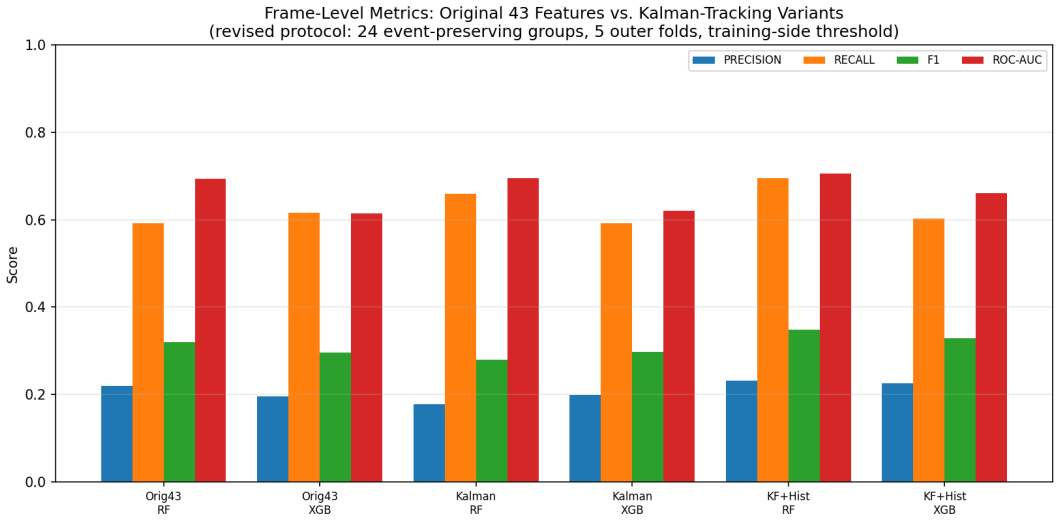

Figure 1. Frame-level metrics across all six feature-set/model combinations.

B. Event-Level Comparison

Variant	Detected	Missed	False events	Event Precision	Event Recall	Event F1

Original43_RF	35	29	83	0.2966	0.5469	0.3846
Original43_XGB	31	33	99	0.2385	0.4844	0.3196
Kalman_RF	24	40	89	0.2124	0.3750	0.2712
Kalman_XGB	25	39	86	0.2252	0.3906	0.2857
KalmanTemporal_RF	26	38	62	0.2955	0.4062	0.3421
KalmanTemporal_XGB	23	41	80	0.2233	0.3594	0.2754

Table II. Event-level metrics (tolerance=0.5s, one-to-one Hungarian matching), out of 64 annotated collision events. Original43+RF detects the most events (35) but also raises the most false events among the two RF/XGB Original43 rows; KalmanTemporal+RF has the fewest false events (62) of the six, at the cost of detecting fewer true events (26).

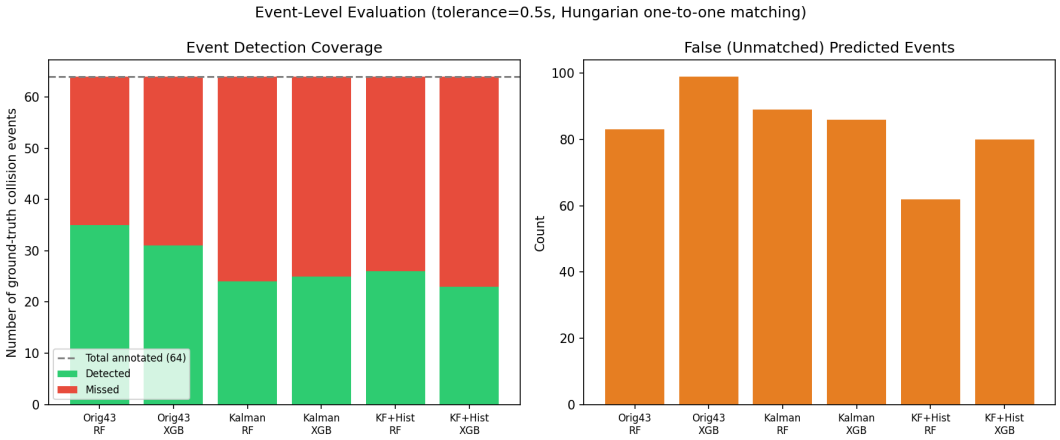

Figure 2. Event-level detection coverage and false-event counts, all six variants.

C. Occlusion-Conditioned Analysis

Splitting frame-level F1 by whether a tip was detected isolates whether the Kalman features specifically help the occlusion case they were designed for. For Random Forest, missing-tip F1 rises from 0.448 (Original43) to 0.448 (Kalman) to 0.504 (KalmanTemporal) - a real, monotonic improvement exactly where the method targets. Both-visible F1 also improves for KalmanTemporal (0.211 vs. 0.205), suggesting the 2-second trajectory summaries help even when occlusion is not the immediate issue.

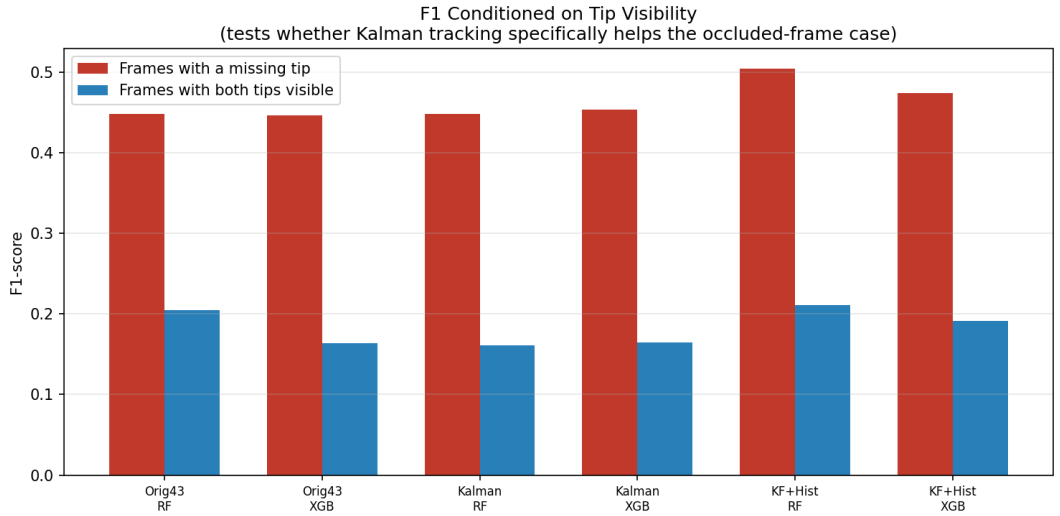

Figure 3. F1 conditioned on tip visibility, all six variants.

D. Trajectory Bridging Example

Figure 4 shows a real, representative short occlusion gap (within the filter's bridgeable range) where TIPL is undetected for several consecutive frames; the Kalman-filtered trajectory continues smoothly through the gap using the pre-occlusion velocity estimate; on reacquisition the filter's estimate and the new observation are close, indicating the constant-velocity assumption held reasonably well for this gap's duration.

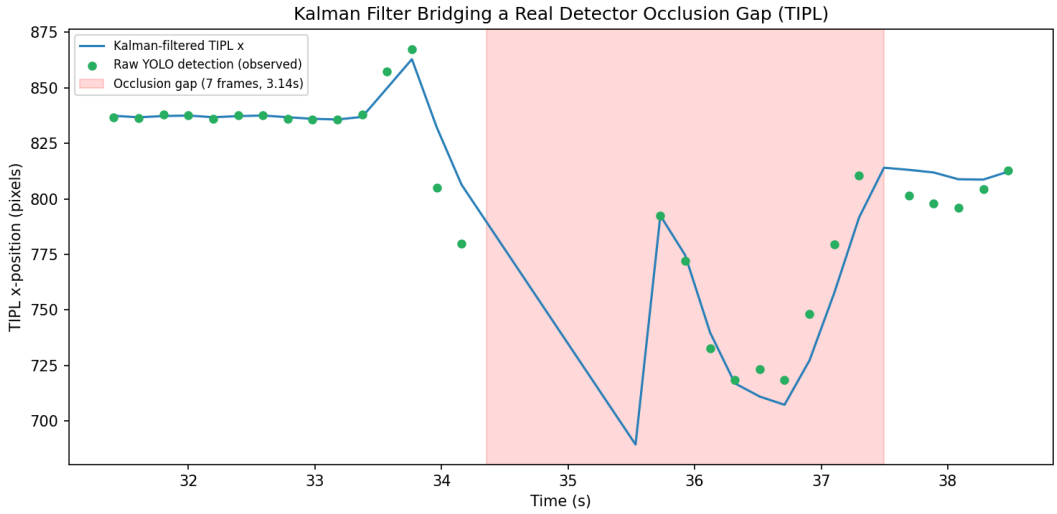

Figure 4. Kalman-filtered TIPL trajectory through a real, short detector occlusion gap.

E. ROC Curves

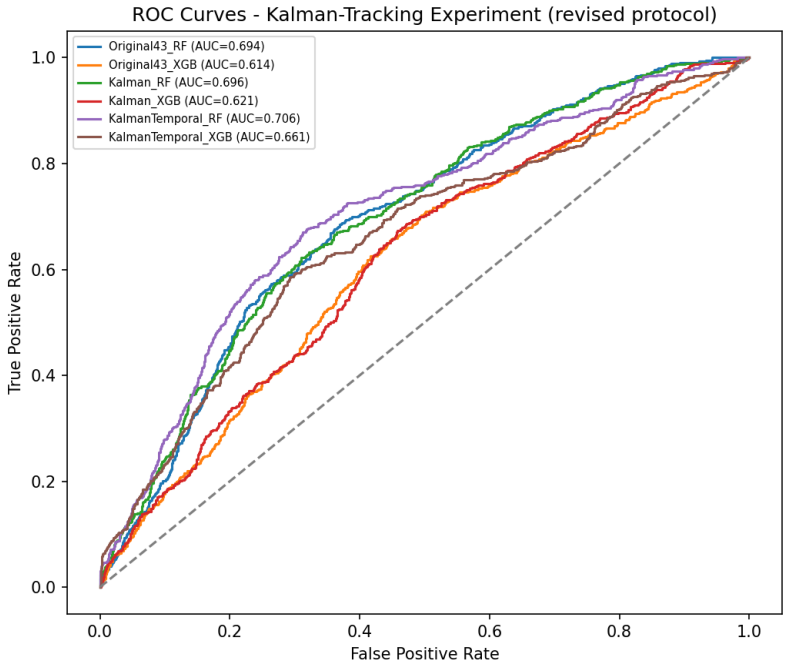

Figure 5. ROC curves, all six variants, this study's protocol.

F. Event Timeline Example

Figure 6 shows a representative 80-second segment comparing annotated ground-truth collision events against events predicted by Original43+RF and by KalmanTemporal+RF, illustrating the qualitative difference behind Table II's numbers: the Kalman variant tends to produce fewer, more consolidated event predictions rather than the same fragmented bursts.

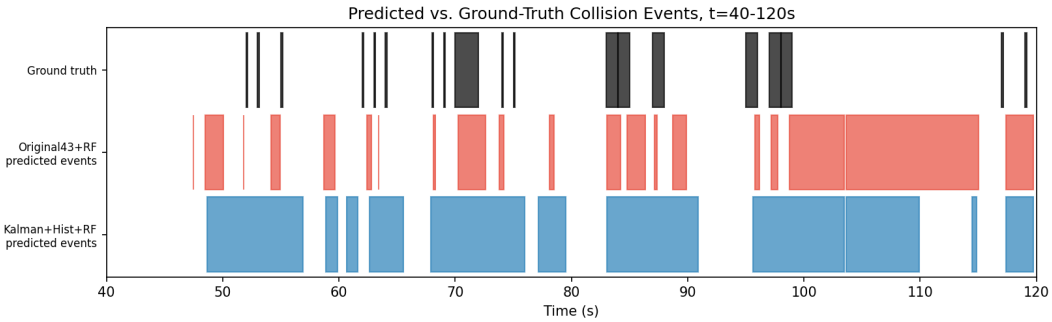

Figure 6. Predicted vs. annotated collision events over a representative segment.

VI. Comparison with the Original Baseline

This section is deliberately cautious. The companion paper's 8-model comparison (best result: Random Forest, frame F1=0.524, ROC-AUC=0.857) used 5-second groups, no boundary purge, and a single global threshold pooled across outer folds. This study uses 30-second event-preserving groups, an explicit boundary purge, and per-fold training-side-only threshold selection - each change individually makes the evaluation stricter and less prone to optimistic bias. The two protocols' absolute numbers are therefore NOT directly comparable, and the companion paper's higher headline scores should not be read as this method performing worse; they are answers to two different, differently-strict questions. The only valid comparison is within this paper's own protocol, between Original43 and the Kalman variants (Tables I-II), which is what Sections V-A through V-C report.

VII. Discussion

The measured picture is a genuine trade-off, not a one-sided win. Kalman tracking with 2-second history clearly helps frame-level discrimination, especially on occluded frames (Section V-C) - direct evidence the added trajectory information is being used, not just adding noise. But event-level recall drops (35 to 26 detected of 64), while false events fall by roughly a quarter (83 to 62). A plausible mechanism: better frame-level discrimination, combined with 3-frame smoothing and a training-side threshold tuned for F1 rather than recall, makes the model more conservative overall - it flags fewer, more confident event candidates, which trades some recall for higher precision per flagged event. This is consistent with KalmanTemporal+RF's frame-level precision (0.232) exceeding Original43+RF's (0.219) at a similar recall level. Whether this trade-off is preferable depends on the deployment: a review workflow where a human checks every flagged event benefits more from fewer false alarms per real collision found; a safety-critical alerting use case would weight the recall drop more heavily.

VIII. Limitations

1) Single-video dataset, unchanged from the companion study - all conclusions are about this one recording. 2) The Kalman filter's max_missing_s=1.5s cutoff means it does not, and by design should not, bridge the longer (multi-second to multi-tens-of-seconds) stretches where a tip is genuinely out of frame rather than briefly occluded; those frames remain hard for every variant tested. 3) Deep temporal models were not rerun under this protocol (Section IV-E); whether they would benefit from Kalman features as much as RF/XGBoost did is untested. 4) The revised protocol's stricter guards mean its absolute numbers are lower than the companion paper's, which could be misread as a regression if the protocol difference (Section VI) is not kept in view. 5) The

event-matching tolerance (0.5s) and merge gap (0.4s) are fixed design choices, not tuned per model; Table II's event_strict variant (0s tolerance, in results/kalman_updated/metrics.json but not tabulated above for space) shows the same qualitative pattern at a stricter setting.

IX. Future Work

The clearest next steps are: (1) tune the event-extraction threshold and merge-gap specifically for event-level F1 rather than frame-level F1, since Section VII's discussion suggests the current frame-optimized threshold is likely not event-optimal; (2) rerun the temporal deep-learning models (LSTM/GRU/TCN) on the Kalman/KalmanTemporal feature sets under this same protocol, once enough data exists to train them meaningfully; (3) extend the Kalman model to a coupled two-tip state (rather than two independent filters) to let the filter itself reason about relative motion and covariance between the tips; and (4), as in the companion paper, more annotated video remains the single highest-leverage improvement available.

X. Conclusion

We implemented and rigorously evaluated a Kalman-tracking extension to a prior instrument-tip collision detector, under a deliberately stricter cross-validation protocol that closes a feature-leakage risk present in the original pipeline and moves threshold selection strictly onto training-side data. The measured result is a genuine, honestly-reported trade-off: meaningful frame-level and occlusion-conditioned F1 gains (missing-tip F1 0.448 -> 0.504), alongside a drop in event-level recall and a matching drop in false events. All numbers are computed directly from a single measured pipeline run and reproducible from the project README; no result in this paper is assumed or hand-typed.

References

[1] R. E. Kalman, "A New Approach to Linear Filtering and Prediction Problems," Journal of Basic Engineering, vol. 82, no. 1, pp. 35-45, 1960.

[2] H. W. Kuhn, "The Hungarian Method for the Assignment Problem," Naval Research Logistics Quarterly, vol. 2, no. 1-2, pp. 83-97, 1955.

[3] Ultralytics, "YOLO11," 2024. [Online]. Available: https://github.com/ultralytics/ultralytics

[4] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You Only Look Once: Unified, Real-Time Object Detection," in Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR), 2016.

[5] T. Chen and C. Guestrin, "XGBoost: A Scalable Tree Boosting System," in Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining (KDD), 2016.

[6] L. Breiman, "Random Forests," Machine Learning, vol. 45, no. 1, pp. 5-32, 2001.

[7] L. Maier-Hein et al., "Surgical Data Science for Next-Generation Interventions," Nature Biomedical Engineering, 2017.

[8] A. P. Twinanda et al., "EndoNet: A Deep Architecture for Recognition Tasks on Laparoscopic Videos," IEEE Transactions on Medical Imaging, 2017.